# HeyTara: Voice-Activated Local Assistant Prototype

SIH26172 – Edge Wake Activator

Prototype implementation and hardware proposal

20 October 2026

**Abstract**

HeyTara explores a hands-free voice path for an assistant that can wait locally for a wake phrase, then capture and transcribe a spoken command. The current implementation is a browser-based software prototype: an AudioWorklet streams microphone audio as 16 kHz PCM to a custom neural wake classifier, keeps a one-second lookback, and opens a WebSocket path only after activation. The receiver saves a WAV recording for local `whisper.cpp` transcription. Synthetic browser fixtures demonstrate the end-to-end plumbing, including a repaired command-truncation defect. The wake model has not met its declared quality gate, and no active neural model is published. The hardware described here is a proposed next step, not a built or measured device.

## 1 Problem and objective

A voice assistant that requires a person to find and tap a screen is inconvenient when their hands or attention are occupied. An always-listening design also needs to avoid sending ordinary room audio away before a user intentionally invokes it. SIH26172 asks for an edge wake activator: detect a chosen phrase locally, preserve the beginning of the following command, and hand that command to recognition.

HeyTara uses "Hey Tara" as the experimental wake phrase. Its intended interaction is simple: the device waits, the person says the phrase and a short command, and the local assistant receives enough audio to transcribe the whole command. This prototype focuses on demonstrating the data path and defining what must be proven before claiming a reliable or resource-constrained edge product.

## 2 Implemented software

The implemented path begins with browser microphone audio. An AudioWorklet resamples the stream from 48 kHz to mono 16 kHz PCM. A local worker evaluates the custom Hey Tara classifier using speech feature encoders and ONNX Runtime Web WASM. The browser retains at most one second of preceding PCM as lookback. Before wake activation, it does not send audio to the receiver. After activation, it sends the retained context and command over a WebSocket. The receiver persists the audio as a PCM16 WAV file and passes the saved recording to a local `whisper.cpp` process for transcription.

This is a saved-WAV handoff, not incremental speech recognition. The one-second lookback is intended to protect the command's first word while wake confirmation completes. A transport defect was found when an utterance exceeded the early socket buffering limit. The repaired implementation bounds pending audio to a complete utterance (up to seven seconds including lookback) and drains queued packets in order after receiver acknowledgement.

The available personal-template workflow first requires three enrolled voice examples. Its microphone path then captures and transcribes after activation. The custom neural path was exercised in an isolated

research QA instance; its failed model is not active in the main app. That frozen research engine uses threshold 0.975 and two consecutive 80 ms scores, with its sensitivity control disabled so the browser cannot silently change the evaluated operating point.

## 3 Evidence and current status

The engineering evidence supports a working synthetic-fixture integration, not a field-ready wake detector. Four preselected synthesized continuous wake-plus-command clips passed through the actual browser worklet, resampler, local WASM worker, WebSocket receiver, saved WAV, and local ASR. Three of four normalized transcripts matched the intended command. In the remaining case, "cabin lights" was heard as "kettle lights"; the same noun error appeared when the original source waveform was transcribed, so this case does not establish a capture truncation defect.

A separate replay caught an actual truncated timer command: an early WAV held only 1.032 seconds and transcribed as "Hey Tara." After the queue and buffer repair, two replays preserved 4.104 seconds and transcribed "Hey Tara, set a timer for five minutes." This is regression evidence for that known synthetic case, not a general latency or accuracy result. Numerical parity checks also confirm that the browser modules reproduce the frozen head and encoder computations within the recorded tolerances.

The classifier is not approved for use as a dependable wake model. The selected frozen research candidate achieved 83.67% aggregate clip recall on its synthetic held-out positive corpus, but only 35.19% clip recall on the 54 external SAPI examples, below the required 80% source gate. All completed V3–V11 quality searches failed their unchanged requirements. The final V11 job completed 30 epochs and six candidate exports on 174,096 training and 5,796 validation contexts; its saved comparison reports no eligible checkpoint. No passing candidate or active neural model can be claimed. Further training and research were stopped at the user's request. The independent primary test remains sealed and unused.

The latest completed software suite reports 81 JavaScript tests and 150 Python tests passing; newer training tests have individual run records. These checks support software behavior and tooling, not human wake accuracy. The feature encoder assets total 2,414,536 bytes, exceeding the 256 KB target by a wide margin. Physical microphone behavior, end-to-end device latency, memory, idle compute and power have not been measured. Synthetic voices and corpus-level false-activation estimates do not predict deployment performance.

## 4 Why it is useful

The prototype makes the intended interaction tangible and exercises the difficult boundaries between continuous audio, wake confirmation, pre-roll, transport, file persistence and local ASR. Keeping the initial buffer in the browser until activation supports a privacy-oriented design goal, while local transcription demonstrates a path that need not depend on a cloud recognition service. The repaired complete-utterance queue addresses the practical requirement that the command remain intact even when wake confirmation and receiver acknowledgement are delayed.

For a user, the potential benefit is hands-free invocation and a short spoken command. For engineering teams, the browser build is a convenient integration harness: it can expose capture and transport failures before a device is selected. These are prototype benefits. The failed wake quality gate means the current system should not be represented as reliable for real users or as satisfying the MCU size target.

# 5 Proposed hardware path

The near-term hardware demonstrator should use a Raspberry Pi 5 or comparable single-board computer (SBC), a USB microphone or a documented I2S microphone, local storage, and the existing local ASR concept. This would move capture and processing onto a dedicated device while retaining enough compute headroom for experiments. It is a desktop-class prototype direction, not an edge-MCU implementation; it does not solve the asset-size target or establish performance. No physical prototype, latency, memory or power result is claimed.

A separate ESP32-S3 with TensorFlow Lite Micro (TFLM) is a possible later low-cost path. It requires a compact classifier that passes human-data quality gates, a compatible exported model and frontend, memory accounting, and board validation. The existing firmware directory contains a reference log-mel frontend and inference adapter only; it is not complete ESP32 firmware. Its 40-bin DS-CNN frontend is not the browser candidate's 32-bin, 96-dimensional embedding pipeline. Porting the current encoder is not established, and the 2.4 MB of feature assets already exceeds the 256 KB goal. Therefore the ESP32-S3 path is explicitly future research, not a current implementation.

## 5.1 Indicative planning BOM

The following INR ranges are unsourced planning allowances for rough budgeting, not supplier quotes or current retail prices. Select a board and microphone only after checking availability, interfaces and the actual demo needs. Low and high totals are the sums of the itemized bounds.

| Item | Qty | Low (INR) | High (INR) |
|---|---|---|---|
| Raspberry Pi 5 / comparable SBC board | 1 | 4,500 | 8,000 |
| USB or I2S microphone module | 1 | 300 | 1,200 |
| MicroSD / local storage | 1 | 400 | 900 |
| Suitable power supply | 1 | 700 | 1,500 |
| Enclosure and cables | 1 set | 300 | 800 |
| Optional speaker for spoken output | 1 | 300 | 1,000 |
| Total including optional speaker | | 6,500 | 13,400 |
| Total without optional speaker | | 6,200 | 12,400 |

These totals exclude display, shipping, taxes, peripherals and any replacement or upgrade parts. The optional speaker is not required for a transcription-only demonstration.

# 6 Bench plan and acceptance evidence

The next bench phase should first run the browser software on the SBC with a fixed microphone, sample-rate and gain configuration. Preserve hashes and versions for the model, frontend and application. Replay saved PCM vectors to compare classifier outputs, then exercise live capture with commands spoken immediately after the wake phrase and with varied acknowledgement delays. Confirm the saved WAV sample count and order, and check that the first command word survives. Record ASR output verbatim, including mistakes; do not repair transcripts with command-specific hints.

Wake quality should be evaluated only after training and calibration produce a candidate that passes the predeclared validation gates. Use speaker- and session-disjoint human wake recordings, representative negatives, quiet and noisy conditions, and long uninterrupted negative exposure. Report event recall by source and condition and false activations per hour with exposure duration. Keep the sealed primary test

untouched until a candidate and policy are frozen. For the physical device, record keyword-end-to-first-received-audio latency using synchronized timestamps, peak RAM, storage use, busy-time percentage and power draw. Publish board, microphone, clock, software/model hashes, and all failed as well as passed runs.

For the later ESP32-S3 evaluation, generate frontend and model parameters from the same approved export, verify host parity, compile against a pinned TFLM revision, and measure arena use and operator support before adding microphone capture. Then validate I2S alignment, scheduling, cooldown, re-arm behavior, buffer bounds, and full-device RAM and power. Passing software tests or a non-acoustic frontend check alone cannot establish wake-word quality or hardware acceptance.

## 7 References and project evidence

Local project records provide the detailed evidence and caveats: browser flow and transport QA, readiness and model quality report, and physical-device handoff and firmware status. The browser integration uses whisper.cpp for local transcription and ONNX Runtime Web WASM for local inference. These links identify project components; they do not imply hardware validation or model approval.